Projet échecs en Python — plan d'apprentissage et échéancier

Version du 10 octobre 2026

Pourquoi ce projet

Un jeu d'échecs à deux, public sur GitHub, qui prouve en quatre jalons que vous savez livrer du Python professionnel : du moteur de règles jusqu'à un pipeline de données.

Sept ans de Smalltalk ne valent pas zéro. Smalltalk a vu naître les tests unitaires (SUnit, ancêtre de JUnit et pytest), le refactoring outillé et le modèle MVC. Votre vrai capital, c'est la conception objet, les tests et la maintenance d'un gros système métier ; ce qui manque, c'est la preuve visible que ce savoir-faire se transpose dans un écosystème courant.

Ce que le projet doit montrer à un recruteur :

- **Du Python idiomatique et testé** (TDD, typage, intégration continue) : le moteur de règles.
- **Un backend réaliste** (API, base de données, migrations de schéma) : le serveur de jeu.
- **Un vrai pipeline de données** sur des millions de parties réelles : le volet Lichess, celui qui parle directement aux recruteurs data engineering.
- **Le travail en équipe** : pull requests, revues de code, historique Git propre.

Le front-end (React ou autre) viendra après ; il n'est pas dans ce plan.

Comment on travaille à deux

Pas de dates : le projet avance de session en session, et chaque session fixe le lot de travail de chacun jusqu'à la suivante.

Une session (60 à 90 min, en visio ou en vrai) suit toujours le même déroulé :

1. Démo : chacun montre ce qu'il a terminé, en le faisant tourner (15 min).
2. Revue croisée des pull requests encore ouvertes, puis merge (20 min).
3. Programmation en binôme sur le point le plus difficile de la prochaine étape (30 min).
4. Découpage du lot suivant : qui fait quoi, et le critère « fini quand » de chaque tâche (15 min).

Entre deux sessions, chacun avance seul, à son rythme, sur sa branche Git. Chaque tâche finie part en pull request ; l'autre la relit quand il peut, sans attendre la session.

On cale la session suivante quand les deux lots sont finis, ou dès que quelqu'un est bloqué. Garde-fou : si trois semaines passent sans session, un appel de 15 minutes suffit pour réduire le lot plutôt que de laisser le projet s'éteindre.

Les règles du dépôt :

- Rien n'arrive sur `main` sans pull request relue par l'autre.
- Pas de code sans tests ; la CI (lint + tests) doit être verte avant le merge.
- Chaque tâche est une *issue* GitHub, rangée dans un tableau *GitHub Projects* (À faire, En cours, Fini) : c'est votre mémoire commune entre deux sessions.
- Code, noms et messages de commit en anglais : c'est ce que liront les recruteurs.

Stack technique

Une stack volontairement standard : chaque outil est un mot-clé qu'on retrouve dans les offres Python backend et data engineering.

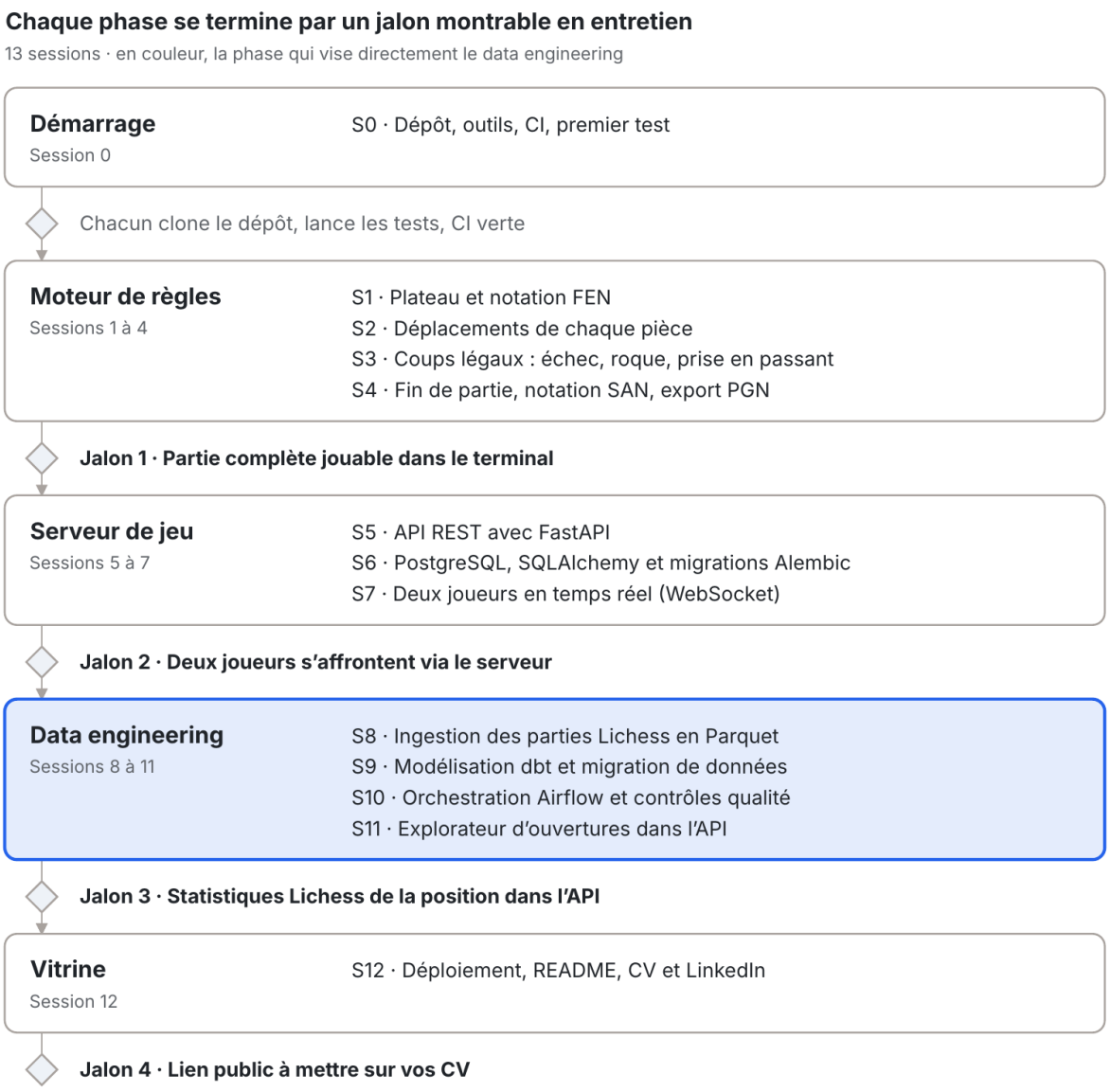
Besoin	Outil	Pourquoi
Langage	Python 3.13 ou plus récent	Annotations de type modernes, messages d'erreur clairs
Projet et dépendances	uv, `pyproject.toml`	Un seul outil pour l'environnement virtuel, les dépendances et le lancement
Qualité du code	ruff (lint + formatage), mypy	Standards du marché ; mypy rattrape les erreurs qu'un langage dynamique laisse passer
Tests	pytest, Hypothesis	pytest est la norme ; Hypothesis génère des positions au hasard pour tester des propriétés
Oracle de test	python-chess (tests uniquement)	Comparer vos coups générés aux siens, jamais pour remplacer votre moteur
API	FastAPI, Pydantic, Uvicorn	Rapide à prendre en main, validation des données et documentation OpenAPI automatiques
Base de données	PostgreSQL, SQLAlchemy 2.0, Alembic	Alembic versionne les migrations de schéma : le cœur du métier de migration de données
Environnement	Docker, Docker Compose	Une commande pour lancer Postgres et l'API chez chacun
Intégration continue	GitHub Actions	Lint et tests à chaque pull request
Traitement de données	Polars, DuckDB, Parquet	Des millions de parties sur un simple portable, sans cluster
Transformation	dbt (adaptateur dbt-duckdb)	Le standard pour les transformations SQL testées et documentées
Orchestration	Apache Airflow	L'orchestrateur le plus déployé ; Dagster est l'alternative plus moderne

En 2026, Airflow reste l'orchestrateur le plus répandu, dbt est devenu le standard de transformation, et DuckDB traite sur une seule machine des volumes de quelques centaines de Go (Data Vidhya, juin 2026). Spark et les plateformes cloud (Snowflake, Databricks) restent hors projet : à aborder ensuite si les offres que vous visez l'exigent.

Échéancier par sessions

13 sessions en 5 phases. À une session toutes les deux à trois semaines, comptez 6 à 9 mois au total, et 2 à 3 mois pour le premier jalon montrable. Le dépôt est public dès la session 0 : chaque jalon peut être cité en entretien sans attendre la fin.

Taille des lots (estimations à revoir après deux sessions) : **lot principal** 5 à 8 h de travail perso, **lot léger** 2 à 4 h, toujours découpés en tâches de 30 à 60 minutes.



Feuille de route · 5 phases, 13 sessions, 4 jalons

Le data engineering arrive après le serveur de jeu, mais il peut passer juste après le jalon 1 si les offres data deviennent prioritaires ; seule la migration Alembic de la session 9 attendra alors la session 6.

Avant la session 0 (chacun de son côté)

- ☐ Installer Python 3.13+, uv, Git et un éditeur (VS Code ou PyCharm)
- ☐ Lire les chapitres 3 à 9 du tutoriel officiel Python (environ 5 h)
- ☐ Faire 5 à 10 exercices de la piste Python d'Exercism pour se dérouiller

Phase 0 — Démarrage

Session 0 · Mise en route. Ensemble : créer le dépôt GitHub, initialiser le projet avec uv, brancher ruff, mypy et pytest, écrire la CI, fixer les conventions.

- ☐ Principal : squelette du package (`chess_engine/`, `tests/`) et un premier test qui passe
- ☐ Léger : README (but du projet, comment lancer les tests) et modèle de pull request

Fini quand : chacun clone le dépôt, lance `uv run pytest` et voit la CI verte sur une pull request.

Phase 1 — Le moteur de règles (pur Python, aucune dépendance)

Session 1 · Plateau et notation FEN. Ensemble : choisir la représentation du plateau (un tableau 8×8 suffit, pas de bitboards) et esquisser les classes.

- ☐ Principal : classes `Board`, `Square`, `Piece`, `Color` ; lecture et écriture FEN
- ☐ Léger : affichage du plateau dans le terminal (lettres ou symboles Unicode)

Fini quand : 5 positions FEN de référence font l'aller-retour sans perte. *À apprendre :* `dataclass`, `Enum`, `__repr__` et `__eq__`, annotations de type, `pytest.mark.parametrize`.

Session 2 · Déplacements des pièces. Ensemble : définir la classe `Move` et l'interface du générateur de coups.

- ☐ Principal : pièces glissantes (tour, fou, dame), cavalier, roi
- ☐ Léger : pions (avance simple et double, prises), sans prise en passant ni promotion

Fini quand : chaque pièce a ses tests, bords et coins du plateau compris. *À apprendre :* générateurs (`yield`), `itertools`, tuples immuables.

Session 3 · Coups légaux et cas spéciaux. Ensemble : programmation en binôme sur la détection d'échec.

- ☐ Principal : jouer et annuler un coup, détection d'échec, filtrage des coups légaux, roque
- ☐ Léger : prise en passant, promotion, et la commande `perft` (compte les positions atteignables à une profondeur donnée)

Fini quand : perft de la position initiale donne 20, 400, 8 902 et 197 281 aux profondeurs 1 à 4, et la position « Kiwipete » 48, 2 039 et 97 862 aux profondeurs 1 à 3 (valeurs de référence). *À apprendre :* `cProfile` pour trouver ce qui est lent, copie ou mutation d'objets.

Session 4 · Fin de partie et notation. Ensemble : la boucle de jeu en ligne de commande.

- ☐ Principal : mat, pat, règle des 50 coups, triple répétition, matériel insuffisant
- ☐ Léger : notation SAN (lecture et écriture) et export PGN

Fini quand : vous jouez une partie complète l'un contre l'autre dans le terminal et exportez son PGN : **jalon 1**. *À apprendre :* expressions régulières (`re`), Typer ou `argparse` pour la ligne de commande.

Phase 2 — Le serveur de jeu

Session 5 · API REST. Ensemble : dessiner les routes et les schémas d'échange.

- ☐ Principal : FastAPI — créer une partie, lire son état, jouer un coup (erreur claire si le coup est illégal)
- ☐ Léger : tests de l'API avec le client de test de FastAPI

Fini quand : une partie complète se joue depuis la documentation interactive (Swagger UI). *À apprendre :* Pydantic, injection de dépendances, bases d'`async` / `await`.

Session 6 · Persistance. Ensemble : modéliser les tables (parties, coups, joueurs).

- ☐ Principal : PostgreSQL via Docker Compose, modèles SQLAlchemy 2.0, couche d'accès aux données
- ☐ Léger : Alembic — première migration et script de données de démo

Fini quand : redémarrer le serveur ne perd aucune partie. *À apprendre :* SQL (jointures, index, transactions), ORM contre SQL brut, migrations versionnées.

Session 7 · Deux joueurs en temps réel. Ensemble : le protocole des messages WebSocket.

- ☐ Principal : WebSocket — chaque joueur reçoit le coup de l'autre en direct
- ☐ Léger : identification simple (pseudo + jeton par partie) et contrôle du tour de jeu

Fini quand : deux clients en ligne de commande s'affrontent via le serveur : **jalon 2**.

Phase 3 — Data engineering sur les parties Lichess

Session 8 · Ingestion. Ensemble : lire un fichier PGN compressé en flux, sans tout charger en mémoire.

- ☐ Principal : télécharger un mois ancien de la base Lichess, le décompresser en flux, parser les PGN par lots, écrire en Parquet
- ☐ Léger : rejouer un échantillon de parties avec votre moteur ; chaque coup refusé est un bug à corriger

Fini quand : un mois de parties est en Parquet, avec un rapport du nombre de parties rejouées sans erreur. *À apprendre :* traitement en flux, Polars, format Parquet.

Session 9 · Modélisation et migration. Ensemble : le schéma analytique (parties, joueurs, ouvertures).

- ☐ Principal : modèles dbt sur DuckDB, avec tests (unicité, valeurs non nulles, valeurs autorisées)
- ☐ Léger : une vraie migration de données sur la base du jeu — ajouter une colonne (par exemple la FEN finale), remplir l'historique, tester le retour arrière

Fini quand : `dbt build` passe et la migration Alembic monte et redescend sans perte.

Session 10 · Orchestration et qualité. Ensemble : dessiner le DAG du pipeline.

- ☐ Principal : DAG Airflow mensuel (téléchargement, parsing, chargement, transformations) ; relancer un mois donne le même résultat
- ☐ Léger : contrôles qualité (volumes, Elo plausibles, doublons) et alerte en cas d'échec

Fini quand : ajouter un nouveau mois tient en une commande, sans doublon.

Session 11 · Valoriser les données. Ensemble : choisir trois questions auxquelles les données doivent répondre.

- ☐ Principal : endpoint « explorateur d'ouvertures » : pour une position, les coups joués et le taux de victoire par tranche d'Elo
- ☐ Léger : notebook d'analyse (ouvertures populaires par niveau, avantage des blancs) avec graphiques dans le README

Fini quand : le serveur de jeu renvoie les statistiques Lichess de la position en cours : **jalon 3**.

Phase 4 — Vitrine

Session 12 · Mise en ligne. Ensemble : répéter une démo de 5 minutes, comme en entretien.

- ☐ Principal : déploiement sur un petit serveur ou une plateforme d'hébergement, schéma d'architecture dans le README
- ☐ Léger : chacun rédige ce qu'il a appris et met à jour CV et LinkedIn

Fini quand : un lien public figure sur vos deux CV : **jalon 4**.

Ensuite, au choix : le front-end React, une petite IA de jeu (minimax), ou Spark et le cloud pour approfondir le volet data.

Plan d'apprentissage : Python pour un développeur Smalltalk

Votre bagage objet se transpose presque entièrement ; l'effort porte sur l'outillage et les idiomes. On apprend juste à temps : chaque session indique ce qu'il faut voir (*À apprendre*), et les ressources sont en fin de document.

En Smalltalk	En Python
`xs do: aBlock`	`for x in xs:`
`collect:`	compréhension de liste `[f(x) for x in xs]`
`select:` et `reject:`	`[x for x in xs if cond]` et `if not cond`
`detect:ifNone:`	`next((x for x in xs if cond), default)`
`inject:into:`	`sum()`, `functools.reduce()`
Blocs	`lambda` (une seule expression) ou fonction imbriquée
`printOn:`	`__repr__` et `__str__`
`=` et `hash`	`__eq__` et `__hash__`, ou `@dataclass(frozen=True)`
`nil`	`None`
`ifTrue:ifFalse:`	`if` / `else`, ou `a if cond else b`
`ReadStream`, `WriteStream`	itérateurs, générateurs (`yield`), `io.StringIO`
`doesNotUnderstand:`	`__getattr__`, à utiliser avec modération
SUnit	pytest
Image et System Browser	fichiers `.py`, modules, Git, IDE

Ce qui change vraiment :

- **Pas d'image.** Le code vit dans des fichiers et l'état disparaît à chaque lancement ; le REPL et le débogueur de l'IDE remplacent l'inspecteur, et des tests rapides deviennent votre boucle de feedback.
- **Environnements et dépendances** (environnement virtuel, `pyproject.toml`, uv) : rien d'équivalent en Smalltalk, c'est ce qui déroute le plus au début.
- **Typage optionnel.** Annotez tout et laissez mypy vérifier : c'est la norme dans le code professionnel récent.
- **Les exceptions ne reprennent pas.** Pas de `resume:` ni de `retry` : une exception remonte la pile jusqu'au `try` / `except` qui la rattrape.
- **Tout n'est pas un message.** `if`, `for` et `while` sont des instructions du langage ; `self` est explicite dans chaque méthode, et l'indentation fait partie de la syntaxe.

Volet data engineering : les parties Lichess

La base ouverte de Lichess publie chaque mois toutes les parties jouées sur le site, en PGN compressé (zstd), sous licence CC0 : usage libre, même commercial, sans autorisation. C'est la matière idéale pour un pipeline qui ressemble à ceux qu'on vous demandera de construire.

Le pipeline, de la source aux statistiques :

1. **Extraction** : télécharger un fichier mensuel `.pgn.zst` et vérifier sa somme SHA256.
2. **Parsing en flux** : décompresser à la volée et lire les parties une à une avec un générateur.
3. **Validation** : rejouer les coups avec votre propre moteur ; les parties rejetées partent dans une table d'erreurs.
4. **Stockage brut** : écrire en Parquet, partitionné par mois.
5. **Transformation** : modèles dbt sur DuckDB (parties, joueurs, ouvertures, positions), avec tests.
6. **Orchestration** : un DAG Airflow mensuel et idempotent qui enchaîne les étapes 1 à 5.

7. **Exposition** : l'explorateur d'ouvertures dans l'API du jeu.

L'étape 3 relie les deux moitiés du projet : votre moteur devient le validateur de données, et des millions de parties réelles deviennent sa batterie de tests. Commencez par un mois de 2013, bien plus petit que les mois récents, et ne passez à l'échelle qu'une fois le pipeline fiable. Pour nommer les ouvertures, Lichess maintient aussi un jeu de données d'ouvertures à joindre à vos parties.

Répartition du travail et temps disponible

Celui qui a le moins de temps sur la période prend le lot léger ; les rôles s'échangent dès que les disponibilités changent.

- **Le lot principal est le chemin critique** : c'est lui qui conditionne la session suivante. Il revient à celui qui a le plus de temps (sans doute ton ami au début).
- **Le lot léger est indépendant** : tests, affichage, documentation, cas particuliers. S'il prend du retard, il ne bloque personne.
- **Chacun touche à chaque couche** (moteur, API, données) au moins une fois : en entretien, vous devez tous les deux pouvoir expliquer tout le projet.
- **La revue croisée est systématique** : relire le code de l'autre fait apprendre deux fois plus, pour une fraction du temps.

Avec un nouveau-né, quelques réflexes pour avancer par petites tranches :

- Termine chaque séance de travail en notant la « prochaine petite action » dans l'issue : tu reprends en 2 minutes au lieu de 20.
- Écris le test avant le code : un test rouge te dit exactement où reprendre après une interruption.
- Garde la lecture (un chapitre, une pull request à relire) pour les moments fragmentés sur téléphone, et le code pour les plages d'une heure.
- Une session qui saute ne tue pas le projet : le garde-fou des trois semaines sert à réduire le lot, pas à culpabiliser.

Ressources et prochaines étapes

Ressource	Pour quoi	Quand
Tutoriel officiel Python (en français)	Les bases du langage	Avant la session 0
Exercism, piste Python	Exercices courts, avec retours	Avant la session 0, puis à volonté
Fluent Python, 2e éd., Luciano Ramalho (O'Reilly)	Python idiomatique pour développeurs expérimentés	Phase 1
Python Testing with pytest, 2e éd., Brian Okken	pytest en profondeur	Phase 1
Chess Programming Wiki	Génération de coups, perft	Phase 1
Architecture Patterns with Python (gratuit en ligne)	Conception objet et DDD en Python : votre terrain	Phases 1 et 2
Tutoriel FastAPI	L'API	Session 5
Tutoriel SQLAlchemy 2.0	ORM et SQL	Session 6

Ressource	Pour quoi	Quand
Fundamentals of Data Engineering, Joe Reis et Matt Housley (O'Reilly)	Vue d'ensemble du métier	Phase 3
Documentation dbt et documentation Airflow	Transformation et orchestration	Sessions 9 et 10

Prochaines étapes :

- ☐ Partager ce document avec ton ami et ajuster ensemble l'ordre des phases
- ☐ Faire chacun les devoirs « Avant la session 0 »
- ☐ Caler la session 0

Sources consultées : Data Vidhya, paysage du data engineering en 2026 · Chess Programming Wiki, résultats perft · Base ouverte Lichess.